

The Limitless Stack

A complete AI operating protocol that gets smarter every time you use it.

Seven tools wired into one operating system: **Claude**, **CLAUDE.md**, **Obsidian**, **NotebookLM**, **Pinecone**, **Hub Workspace**, **Paperclip**. This guide takes you from *git clone* to a working stack in roughly 30–45 minutes — and crucially, it explains *how the system compounds learning and self-heals over time*, so the version you run on day 90 is meaningfully smarter than the one you booted on day 1.

Read it once before you start; paste it into your Claude session as the onboarding context so your Claude inherits the same operating discipline this stack runs on.

What you'll have when you're done

- **A working knowledge base** in Obsidian that compounds — every new source updates many pages, never derived from scratch.
- **Semantic recall** across every repo and document via Pinecone — natural-language search, no grep gymnastics.
- **Curated topic notebooks** in NotebookLM, including a special *reminder bucket* your Claude reads at the start of every session.
- **Mechanical session rituals**: Roll Call preflight at start, 8-step checklist at end. Both bundled as installable skills, both enforced by your CLAUDE.md.
- **A living anti-patterns log + a self-healing loop** — institutional memory of mistakes worth not repeating, plus an autonomous diagnose-and-repair pipeline for code bugs.
- **Six skills** auto-installed to `~/ .claude/skills/` by the installer.

The seven tools

Tool	Role	What it owns
Claude	Reasoning engine	Reads, writes, connects, decides
CLAUDE.md	Identity + rules	Trust anchor, voice, guardrails per context
Obsidian	Knowledge base	Structured wiki: architecture, decisions, entities
NotebookLM	Research desk	Deep research across curated source collections
Pinecone	Semantic memory	Full-text recall across every repo and document
Hub Workspace	Multi-model agent runtime	Chat surface — Gemini default, Claude opt-in
Paperclip	Coordination	Org chart, budgets, tickets, approvals

Why this exists — the three problems it solves

1. Amnesia. Every Claude session starts cold. Long contexts balloon costs and degrade reasoning. Classic RAG (chunks → answer → forget) doesn't compound. **The Stack solves this** by giving Claude a persistent memory layer it can read AND write — wiki + Pinecone + NotebookLM — so each session inherits everything prior sessions learned.

2. Drift. Operating rules live in CLAUDE.md, anti-patterns in a synthesis page, recent decisions in the log. Without enforcement, agents stop reading them. **The Stack solves this** with three-layer enforcement: a mandatory-first-action banner in CLAUDE.md, skill descriptions that auto-trigger on greetings, and a Roll Call preflight that mechanically gates substantive work.

3. Bug repair at scale. When you're running many vertical apps on one shared architecture, manual triage doesn't scale. **The Stack solves this** with a self-healing pipeline: in-app capture → Claude diagnostic agent (CLAUDE.md as system prompt) → operator review → sandboxed repair agent → PR → human review → merge → log entry that future sessions learn from.

The flywheel — how the Stack compounds over time

This is the part that most documentation handwaves and most agents fail to grasp. The Limitless Stack isn't a static toolkit; it's a system designed to *get measurably smarter every time you use it*. Here's the mechanism, made concrete.

The four feedback loops

Each session triggers four flywheels that compound knowledge across the stack. They run in parallel and reinforce each other:

[illegible]

Loop 1 — Source ingest

Drop a source in `raw/`. Claude reads it, creates a summary page in `wiki/sources/`, extracts crisp factual claims, identifies entities and concepts. Net effect: one source $\rightarrow$ potentially fifteen wiki pages updated.

What compounds: the more sources you ingest, the more cross-references the next ingest can make. Your Claude is building a graph, not a list.

Loop 2 — Wiki refinement

When two sources contradict, Claude flags it explicitly with a `> [!warning]` callout — never silently overwrites. Substantive query answers get filed back as `wiki/synthesis/<slug>.md`. Periodic lint passes catch orphan pages, broken links, stale claims. **What compounds:** the wiki gets sharper, less contradictory, and more navigable over time. The graph view in Obsidian becomes a real navigation tool, not a pretty visualization.

Loop 3 — Index sync

Step 4 of the end-of-session checklist runs `pinecone-sync.py --changed-only`; Step 5 runs `notebooklm-wiki-refresh.py`; Step 6 verifies it actually landed. Every wiki edit is reflected in semantic search and topic notebooks within minutes of session end. **What compounds:** Pinecone's recall improves with every new chunk; NotebookLM's syntheses get richer with every new source. The next query benefits from work done in every prior session, automatically.

Loop 4 — Anti-patterns + reminder bucket

When a session catches a recurring failure mode, it lands as a numbered entry in `wiki/synthesis/claude-anti-patterns.md`. That file is in the reminder bucket's allowlist, so the next session — which queries the reminder bucket as step 3 of Mandatory First Action — reads the new entry as part of its opening context. **What compounds:** mistakes never get repeated. The system literally learns from itself, session by session.

A worked example: 90 days of compounding

Abstract framing only goes so far. Here's what an actual user's stack looks like at three checkpoints, drawn from real activity in the OpenScaffold vault:

Day 1 — fresh install

- Wiki: 7 starter pages (4 foundational vendored from the install + log/index/overview).
- Pinecone: index empty until first sync.
- NotebookLM: 2 buckets you created (default + reminder), 5 reminder files seeded.
- Anti-patterns: 11 entries inherited from the vendored starter — failure modes other operators paid for so you don't have to.
- Roll Call: returns READY because everything you have is wired correctly.

Day 30 — after a month of regular use

- Wiki: ~80 pages. Each new source averaged 5–7 pages touched (one-source-multi-page-update is the magic).
- Pinecone: ~3,500 chunks indexed across the repos you're tracking. Semantic queries return results with scores routinely ≥ 0.85 .
- NotebookLM: per-project notebooks created for your 3 main projects; reminder bucket has been refreshed 30 times, each time verified.
- Anti-patterns: 14 entries (you added 3 from your own near-misses). Future sessions inherit your team's specific lessons.
- Reminder bucket queries return your team's actual operating rules — not generic ones — within seconds at session start.
- End-of-session checklist completion rate: ~95% (Roll Call's git-status check fires WARN if you ever skip step 1, which catches the rest).

Day 90 — the system is meaningfully different from day 1

- Wiki: ~250 pages. The Obsidian graph view shows clear clusters by domain. Lint passes are short — the wiki self-corrects most contradictions during ingest now because Claude has enough context to cite both sides.
- Pinecone: ~15,000 chunks. "Where do we handle X?" queries return the right files in <2 seconds and the top hit is almost always the canonical implementation.
- NotebookLM: 6+ buckets — default, reminder, plus per-project. Topic-specific synthesis queries return quality on par with manually-written architecture docs because the source set is curated and current.

- Anti-patterns: 20+ entries. You catch new agents falling into known traps within their first session because the reminder bucket surfaces the relevant entry on Roll Call.
- Hub Workspace conversations have history (in the wiki, not the chat surface — chat is per-tab by design). Decisions made in chat get filed back as syntheses; you can re-read "why did we do X?" three months later and find your own reasoning.
- **The big shift:** new contributors ramp in days, not weeks, because the wiki + reminder bucket + skills give them everything an experienced operator would explain.

The compounding is not abstract. It's measurable: page count, chunk count, anti-pattern count, query latency, ramp time. If your numbers aren't moving in the right direction over 90 days, the loop has a leak — usually a skipped end-of-session step. The verification protocol exists to catch those leaks early.

The self-healing loop — autonomous bug repair

The fifth flywheel is bigger than the other four put together. Once your stack is wired up, you can give Claude permission to *fix bugs autonomously* — with CLAUDE.md as the trust anchor, sandboxed execution, and human review before anything merges. It's the closest thing in this stack to magic, and it's the single biggest force multiplier for a small team.

The ten-stage pipeline

- 1. In-app capture.** User hits a bug, clicks the floating bug icon. Description + automatic context bundle (console logs, network requests, recent actions, viewport, route) gets posted to the bug-report endpoint.
- 2. Persistence + diagnostic dispatch.** Server inserts the bug record. A background promise fires Claude with CLAUDE.md as system prompt and the bug context as user message. **Why CLAUDE.md as system prompt:** it's the trust anchor — domain knowledge, conventions, what's safe to touch, what's not.
- 3. Operator review.** Admin view shows: severity, confidence, root cause, suspected files, proposed fix. Operator decides whether to dispatch the repair agent. Cheap diagnostics, expensive repair — separation of concerns.
- 4. Self-heal dispatch.** GitHub repository_dispatch event with bug context and a callback URL fires off the actual repair workflow. Operator approval gates this — autonomous diagnose, human-greenlit repair.
- 5. Workflow initialization.** GitHub Actions runner checks out the repo, creates a branch, installs the agent SDK. Sandboxed — runs in an ephemeral runner, not production.
- 6. Agent investigation loop.** Claude agent runs with a tool whitelist (read, list, search, edit, write, bash, finish). 25-turn budget — no infinite loops. Reads CLAUDE.md, drills into suspected files, makes changes.
- 7. Change detection + commit.** If the agent produced changes, stage and commit. If not, report failure with reasoning.
- 8. PR creation.** Bot opens a PR with full context: original user description, diagnostic finding, agent investigation summary, link to the workflow run for audit.
- 9. Human review + merge.** Standard review process. Auto-merge is OFF by default — every fix gets a human eyeball. Conditional auto-merge can layer on later for trusted patterns.
- 10. Final reconciliation.** Bug report record updated with full trajectory. Wiki gets a log entry. If a new failure pattern emerged, it goes into anti-patterns.md. **Net effect:** the next bug of similar shape gets diagnosed and fixed faster, because the wiki + reminder bucket + anti-patterns now have the lesson.

Self-healing — the architectural principles

Five principles make the loop trustworthy. Skip any of them and you get a system that's faster but less safe, or safer but slower. All five together is the sweet spot.

CLAUDE.md as trust anchor. The diagnostic and repair agents both receive CLAUDE.md as system prompt — never bypass it. It's the place where you encode "don't touch the migrations directory without asking," "never modify the shared users table from the Hub," "this app uses lsh_ table prefix," and so on. The agent inherits the operator's full operational context.

Diagnose before repair. Always run the cheap diagnostic pass first. The repair pass is optional and gated. This separates "figure out what's wrong" (fast, low-risk, frequent) from "actually change code" (slower, higher-risk, gated by operator approval).

Sandboxed execution. The repair agent runs in an ephemeral GitHub Actions runner with a code-level tool whitelist — never on prod, never with broad shell access. The whitelist is enforced in the agent SDK, not just suggested in the prompt.

Human merge by default. PRs, not direct commits. Auto-merge requires explicit per-repo policy. Operators stay in the loop on every fix. The loop is autonomous up to the PR; merging is not.

Closed-loop auditability. Every state transition is recorded on the bug-report record: dispatch, diagnostic, repair, PR, merge, post-merge log entry. Operators can audit the full trajectory and replay any step.

How self-healing compounds with the other loops

Each successful self-heal updates three things: the codebase (via PR), the wiki/log (auto-appended on merge), and — if a new failure mode emerged — `claude-anti-patterns.md`. The next session's Roll Call sees the new commit, the new log entry, and the new anti-pattern via the reminder bucket. Future agents diagnosing similar bugs start with that context already loaded.

Net effect across many bugs: mean-time-to-repair shrinks. Recurring failure patterns get caught at diagnosis. The codebase gets less buggy as the system catches its own anti-patterns and refactors them out via PRs.

Important: self-healing is opt-in. The first time you run the stack, self-heal is not active — you'd need to set up the GitHub Actions workflow, the agent SDK, and the bug-report endpoint per repo. The `self-heal/templates/` directory in the LimitlessStack repo has the canonical files (workflow YAML, agent script, setup guide) you copy into each app repo when you're ready. Start with one app, prove the loop, then roll out.

The six skills — installed by `./install.sh`

Each skill is a self-contained Markdown file Claude reads on demand. The skill's **description** field is what makes it auto-trigger — when Claude sees a phrase that matches the description, the skill loads and Claude follows its instructions. All six skills install to `~/.claude/skills/<name>/SKILL.md`.

Skill	What it does	Triggers on
limitless-stack	Umbrella protocol. 4-tool lookup, mandatory first a	References to <code>limitless-stack</code> , <code>OpenStack</code> , <code>cloud</code> , <code>theaters</code> , <code>to</code>
roll-call	Session-start preflight. Calls <code>limitless-preflight.sh</code> . "Verifies all 7 tools. Returns <code>READY</code> or <code>FLIGHT</code> "	Session-op
notebooklm	Full NotebookLM CLI API. Notebooks, sources, queries	References to <code>NotebookLM</code> , <code>requests</code> , <code>updates</code> , <code>briefings</code> , <code>single</code>
four-tool-lookup	Codifies the wiki → Pinecone → NotebookLM look	Fact discipline. Separable questions about the results of a search
verify-before-claim	Guards against false 'unavailable' claims. Forces	Clarification abouts election on path before declaring it unavailable
karpthy-guidelines	Code-edit discipline. Four named principles: Think	Before, Coding Simply, First, Sober Changes, Goal-Driven

Operational tools shipped alongside the skills

`install.sh` also drops six tools into your vault's `tools/` directory:

`limitless-preflight.sh` — The script Roll Call calls. Validates 7 tools, dedupe-sweeps every NotebookLM bucket, returns 0/1/2.

`session-bootstrap.sh` — Current-thesis snapshot at session start (wiki state, Pinecone count, recent log entries).

`notebooklm-wiki-refresh.py` — Routes wiki files into per-project + default + reminder buckets. Verifies content actually landed (anti-pattern #12 protection).

`notebooklm-dedupe.py` — Sweeps any bucket for duplicates. Used when routing produces ghost duplicates.

`pinecone-sync.py` — Chunks corpus, upserts to index. `--changed-only`, `--dry-run`, `--repo NAME`. Rate-limit aware.

`pinecone-search.py` — Semantic search with optional `--repo` filter.

How the rituals are enforced (three layers)

There's no single magic hook — three layers work together. Understanding all three is the difference between Claude following the protocol by accident and Claude following it reliably.

Layer 1 — CLAUDE.md mandatory-first-action banner

Your vault's CLAUDE.md (seeded by install.sh) opens with:

```
> ■ MANDATORY FIRST ACTION – NO EXCEPTIONS
> Before answering ANY question, making ANY file changes, or starting ANY task:
> 1. Invoke the roll-call skill.
> 2. Run tools/session-bootstrap.sh.
> 3. Query the reminder NotebookLM bucket.
> 4. Read wiki/index.md.
> 5. Do NOT skip these steps.
```

Claude Code and Cowork read CLAUDE.md automatically at session start. The banner is the first thing in the file, with explicit "NO EXCEPTIONS" framing. Claude treats it as a hard precondition. **What you do:** nothing — don't delete the banner. If you customize the file, leave it intact.

Layer 2 — skill descriptions auto-trigger

The roll-call skill description specifically lists trigger phrases: "hey claude", "are you ready", "roll call", "preflight", "is everything connected", any greeting that opens a session. So even if Claude blew past the CLAUDE.md banner (it won't, but defense-in-depth), the moment you say "hey claude" the skill loads and Claude follows its instructions — including the rule "do not begin substantive work until Roll Call returns READY."

Layer 3 — explicit bootstrap prompt (manual backup)

Belt and suspenders. Paste this when you want to be 100% sure:

```
Hey Claude. This vault uses the Limitless Stack protocol. Run roll-call,
then bootstrap, then query my reminder NotebookLM bucket for the current
operating rules and recent anti-patterns. Then read wiki/index.md.
Then we'll start work.
```

After Claude responds, you should see Roll Call output (READY/WARN/BLOCK with per-tool checks). If you don't see that, ask: "did you run Roll Call?" — if not, ask Claude to do so before continuing.

End-of-session — same three-layer model

The 8-step checklist (in CLAUDE.md and limitless-stack SKILL.md) is enforced the same way: banner, skill, and explicit prompt. The phrase that's most reliable: *"Run the end-of-session checklist."* Claude walks through all eight steps, runs the scripts, reports verify_failed counts, and surfaces anything that didn't land.

Pro tip: next session's Roll Call catches most failures from the previous session's wrap. The two rituals are designed to validate each other.

Step-by-step install

Run these in order. Each step has a verification check.

Prerequisites (5 min)

- Accounts: **GitHub** (with org access), **Pinecone** (free tier OK), **Google** (NotebookLM access in Chrome), **invite to LimitlessStack repo**.
- Software on Mac: **Homebrew**, **Python 3.11**, **Node.js**, **Obsidian**, **gh CLI**, **Chrome** with the Claude extension.

1. Clone and install

```
$ gh repo clone Open-Scaffold-Labs/LimitlessStack ~/LimitlessStack
$ cd ~/LimitlessStack
$ ./install.sh ~/your-vault-path
```

Verify: `ls ~/.claude/skills/` shows six skills. `ls ~/your-vault-path/wiki/` shows `index.md`, `log.md`, `overview.md`, plus four foundational pages.

2. Pinecone — Keychain + first sync

```
$ security add-generic-password -a pinecone -s pinecone-api-key -U -w
# (paste key when prompted)

$ cd ~/your-vault-path
$ python3.11 tools/pinecone-sync.py
```

Verify: `python3.11 tools/pinecone-search.py "any test query"` returns results.

3. Create your NotebookLM buckets

In Chrome (signed into NotebookLM), create at minimum two notebooks: **Wiki default** and **Reminder bucket**. Note each notebook's ID. Per-project notebooks are optional but recommended.

4. Wire your bucket IDs into the config

Edit `tools/notebooklm-wiki-refresh.py` — replace Matt's bucket IDs (`NOTEBOOK_ROUTES`, `DEFAULT_ROUTE`, `REMINDER_NOTEBOOK_ID`, `REMINDER_FILES`) with yours. Same for the bucket-ID block in `tools/limitless-preflight.sh`.

5. Customize for your domain + run Roll Call

Edit `~/your-vault-path/CLAUDE.md` — replace `[YOUR DOMAIN]` placeholders. Then verify the whole stack:

```
$ bash tools/limitless-preflight.sh ; echo "exit: $?"
```

Exit 0 = READY · 1 = WARN · 2 = BLOCK.

Customization checklist — disconnect from the worked example

- Bucket IDs** in `tools/notebooklm-wiki-refresh.py` + `tools/limitless-preflight.sh` (operational, must change).

- **CLAUDE.md** — replace [YOUR DOMAIN] placeholders.
- **Wiki entity links** — [[entities/openscaffold]] in foundational pages can stay as worked examples or be replaced.
- **Repo clone list** in raw/ — replace OpenScaffold-specific examples with your own repos.
- **Keep as-is:** all six skills, generic operational tools, vault directory structure.

Day-to-day operating

Ingest a new source

Drop in `raw/`. Ask: *"Ingest the new source at `raw/{filename}`."* Claude reads it, creates a source page, updates entity/concept pages, flags contradictions, updates index, appends log entry.

Ask a question

Just ask. Four-tool lookup runs automatically. Substantive answers get filed back as `wiki/synthesis/<slug>.md`.

Periodic lint

Every few weeks: *"Run a lint pass and write the report."* Surfaces contradictions, stale claims, orphans, broken links, coverage gaps, index drift.

Catch a new failure mode

Append numbered entry to `wiki/synthesis/claude-anti-patterns.md`. Format: trigger pattern, why-it's-tempting, why-it's-wrong, corrective rule. Future sessions read it via the reminder bucket.

Schedule nightly Pinecone sync

`crontab -e` running `tools/pinecone-sync.py --changed-only` overnight. Keeps the index current without ceremony.

Reference card

Foundational reading (in `obsidian/vault-template/wiki/`)

- `concepts/llm-wiki-pattern.md` — Karpathy's LLM Wiki pattern. The methodology.
- `sources/claude-code-karpathy-obsidian-video-2026-04-14.md` — Jack Roberts video that distilled the four-tool framing.
- `concepts/notebooklm-workflow.md` — The seven-bucket routing pattern.
- `synthesis/claude-anti-patterns.md` — Living failure-mode log. 11+ entries on day one.

Where to dig deeper

- `skills/limitless-stack/SKILL.md` — full protocol, every section, 400+ lines.
- `docs/setup-guide.md` — detailed step-by-step.
- `self-heal/templates/` — workflow YAML, agent script, setup guide for the self-healing pipeline.
- `README.md` — what's in the repo.
- Upstream: `forrestchang/andrej-karpathy-skills` for karpathy-guidelines source.

The Limitless Stack is one integrated system that gets meaningfully smarter every time you use it. Don't try to use just one tool — the value compounds across all seven, across every session, across every bug fix and every source ingest. Welcome aboard.